

使用 Apache Iceberg 和 BigLake 建構 AI 專用的整合式資料湖倉

來源：<https://codelabs.developers.google.com/iceberg-biglake-ai-lakehouse?hl=zh-tw>

步驟數：9

透過 Apache Iceberg 和 BigLake 探索公開資料集，並使用 BigQuery ML 和物件資料表進行 AI 作業。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 連線至 Apache Iceberg 公開目錄](#)
4. [4. 查詢結構化 Iceberg 資料](#)
5. [5. Apache Iceberg 中的時間旅行](#)
6. [6. 使用 BigQuery ML 進行結構化 AI](#)
7. [7. 搭配 BigLake 使用 Unstructured AI](#)
8. [8. 清除所用資源](#)
9. [9. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

在本程式碼研究室中，您將探索 Google Cloud 上的 Unified Data Lakehouse 功能。您將透過 [BigLake](#) 上的 **Apache Iceberg REST Catalog** 存取公開資料集，然後將 Google Cloud 的 AI 功能套用至結構化和非結構化資料。

您將使用 Apache Iceberg 查詢紐約市計程車經典資料集，深入瞭解**時空旅行**來稽核資料變更，然後使用 [BigQuery ML](#) 和 [Gemini](#) 對資料執行 AI 模型。

學習內容

- 使用 [Google Cloud Serverless for Apache Spark](#) 查詢 BigLake 上託管的 **Apache Iceberg** 公開資料集。
- 查詢 Apache Iceberg 格式的結構化資料。
- 示範 Apache Iceberg 中的「時間旅行」。
- 使用 **BigQuery ML** 訓練結構化資料的預測模型。
- 建立 **BigLake** 物件資料表 (非結構化資料)，並使用 **Gemini** 分析圖片。

軟硬體需求

- 網路瀏覽器，例如 [Chrome](#)。
- 已啟用計費功能的 Google Cloud 專案。

預計費用和時間長度

- **完成時間**：約 45 分鐘。
 - **預估費用**：低於 \$2.00 美元。我們使用公開資料集和無伺服器查詢，以維持低成本。
-

步驟 2 · 約 5 分鐘

2. 設定和需求

在這個步驟中，您將準備環境並啟用必要的 API。

啟動 Cloud Shell

您將透過 [Google Cloud Shell](#) 執行大部分指令。

1. 按一下 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 驗證：

□□□

```
gcloud auth list
```

3. 確認專案：

□□□

```
gcloud config get project
```

4. 如果未設定專案，請使用專案 ID 設定：

□□□

```
gcloud config set project <YOUR_PROJECT_ID>
```

啟用 API

執行下列指令，為 [BigQuery](#)、[Cloud Resource Manager](#) 和 [Vertex AI](#) 啟用必要 API：

□□□

```
gcloud services enable \  
  bigquery.googleapis.com \  
  aiplatform.googleapis.com \  
  cloudresourcemanager.googleapis.com
```

設定環境並建立依附元件 bucket

1. 在終端機中設定環境變數：

```
export PROJECT_ID=$(gcloud config get project)  
export REGION=us-central1  
export DEPS_BUCKET=$PROJECT_ID-deps-bucket
```

2. 建立依附元件 Cloud Storage bucket。提交工作時，PySpark 指令碼會上傳至此處：

```
gcloud storage buckets create gs://$DEPS_BUCKET --location=$REGION
```

3. 連線至 Apache Iceberg 公開目錄

在這個步驟中，您將連線至 Google Cloud BigLake 上託管的實際工作環境級別 Apache Iceberg 目錄。

使用 Serverless for Apache Spark Batch CLI 執行 Spark SQL

我們將使用 [Google Cloud Serverless for Apache Spark](#) 執行 PySpark 工作，無須管理基礎架構。我們會將其設為指向公開的 BigLake REST 目錄。

1. 定義 BigLake REST 目錄屬性，避免重複設定。這項設定會告知 Spark：

- 使用 `iceberg-spark-runtime` 和 `iceberg-gcp-bundle` 程式庫。
- 使用 **BigLake REST** 目錄端點設定名為 `my_catalog` 的目錄。
- 使用 **Google Cloud Storage (GCS)** 讀取資料檔案，而非預設的本機檔案系統。
- 將這個 `my_catalog` 目錄設為工作階段的預設目錄。
- 使用供應的憑證，提高安全性並簡化資料存取權。

```
export METASTORE_PROPERTIES="^|^spark.jars.packages=org.apache.iceberg:iceberg-spark-
runtime-3.5_2.12:1.10.0,org.apache.iceberg:iceberg-gcp-bundle:1.10.0|\
spark.sql.catalog.my_catalog=org.apache.iceberg.spark.SparkCatalog|\
spark.sql.catalog.my_catalog.type=rest|\
spark.sql.catalog.my_catalog.uri=https://biglake.googleapis.com/iceberg/v1/restcatalog|\
spark.sql.catalog.my_catalog.warehouse=gs://biglake-public-nyc-taxi-iceberg|\
spark.sql.catalog.my_catalog.io-impl=org.apache.iceberg.gcp.gcs.GCSFileIO|\
spark.sql.catalog.my_catalog.header.x-goog-user-project=$PROJECT_ID|\
spark.sql.catalog.my_catalog.header.X-Iceberg-Access-Delegation=vended-credentials|\
spark.sql.catalog.my_catalog.rest.auth.type=org.apache.iceberg.gcp.auth.GoogleAuthManager
|\
spark.sql.defaultCatalog=my_catalog|\
spark.sql.extensions=org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions|\
spark.log.level=ERROR"
```

2. 建立簡單的測試查詢檔案：

```
cat <<EOF > test.py
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

spark.sql("SHOW TABLES IN public_data").show()
EOF
```

3. 提交批次工作：

```
gcloud dataproc batches submit pyspark \  
  --project=$PROJECT_ID \  
  --region=$REGION \  
  --version=2.3 \  
  --properties="$METASTORE_PROPERTIES" \  
  --deps-bucket=gs://$DEPS_BUCKET \  
  test.py
```

等待批次工作完成執行。工作完成後，您應該會看到類似以下的輸出：

```
+-----+-----+-----+  
| namespace |      tableName | isTemporary |  
+-----+-----+-----+  
| public_data |      nyc_taxicab |      false |  
| public_data | nyc_taxicab_2021 |      false |  
+-----+-----+-----+
```

步驟 4 · 約 5 分鐘

4. 查詢結構化 Iceberg 資料

連線完成後，您就能透過 SQL 完整存取資料集。我們將查詢以 Iceberg 資料表為模型的紐約市計程車資料集。

執行標準匯總查詢

建立名為 `query.py` 的檔案：

```
cat <<EOF > query.py
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

query = """
SELECT
    passenger_count,
    COUNT(1) AS num_trips,
    ROUND(AVG(total_amount), 2) AS avg_fare,
    ROUND(AVG(trip_distance), 2) AS avg_distance
FROM public_data.nyc_taxicab
WHERE data_file_year = 2021 AND passenger_count > 0
GROUP BY passenger_count
ORDER BY num_trips DESC
"""

spark.sql(query).show()
EOF
```

然後使用 Serverless for Apache Spark 提交：

```
gcloud dataproc batches submit pyspark \
  --project=$PROJECT_ID \
  --region=$REGION \
  --version=2.3 \
  --properties="$METASTORE_PROPERTIES" \
  --deps-bucket=gs://$DEPS_BUCKET \
  query.py
```

請稍候幾分鐘，等待批次工作執行完畢。

工作完成後，您應該會看到類似以下的輸出：


```
+-----+-----+-----+-----+
|passenger_count|num_trips|avg_fare|avg_distance|
+-----+-----+-----+-----+
|           1| 21508009|   18.82|         3.03|
|           2|  4424746|   20.22|         3.40|
|           3| 1164846|   19.84|         3.27|
|           5|   718282|   18.88|         3.07|
|           4|   466485|   20.61|         3.44|
|           6|   452467|   18.97|         3.11|
|           7|         78|   65.24|         3.71|
|           8|         49|   57.39|         5.88|
|           9|         35|   73.26|         6.20|
|          96|          1|   17.00|         2.00|
|         112|          1|   15.00|         2.00|
+-----+-----+-----+-----+
```

為什麼要在此使用 Apache Iceberg ？

- **分區修剪**：查詢會依 `data_file_year = 2021` 進行篩選。Iceberg 可讓引擎完全略過掃描其他年份的資料。
 - **引擎靈活性**：您可以在 Spark、Trino 或 BigQuery 中執行這項作業，不必複製資料！
-

步驟 5 · 約 5 分鐘

5. Apache Iceberg 中的時間旅行

Iceberg 最強大的功能之一是「時空旅行」。您可以查詢過去版本或快照中的資料。

查看資料表記錄

建立名為 `history.py` 的檔案：

```
cat <<EOF > history.py
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

spark.sql("SELECT * FROM public_data.nyc_taxicab.history").show()
EOF
```

然後提交：

```
gcloud dataproc batches submit pyspark \
  --project=$PROJECT_ID \
  --region=$REGION \
  --version=2.3 \
  --properties="$METASTORE_PROPERTIES" \
  --deps-bucket=gs://$DEPS_BUCKET \
  history.py
```

控制台應會顯示類似下列內容的輸出結果：

made_current_at	snapshot_id	parent_id	is_current_ancestor
2026-01-07 21:32:...	6333415779680505547	NULL	true
2026-01-07 21:34:...	1840345522877675925	6333415779680505547	true
2026-01-07 21:36:...	7203554539964460256	1840345522877675925	true
2026-01-07 21:38:...	4573466015237516024	7203554539964460256	true
2026-01-07 21:40:...	3353190952148867790	4573466015237516024	true
2026-01-07 21:42:...	1335547378580631681	3353190952148867790	true
2026-01-07 21:44:...	8203141258229894239	1335547378580631681	true
2026-01-07 21:46:...	1597048231706307813	8203141258229894239	true
2026-01-07 21:48:...	6247811509231462655	1597048231706307813	true
2026-01-07 21:50:...	2527184310045633322	6247811509231462655	true
2026-01-07 21:52:...	2512764101237223642	2527184310045633322	true
2026-01-07 21:52:...	7045957533358062548	2512764101237223642	true
2026-01-07 21:53:...	531753237516076726	7045957533358062548	true
2026-01-07 21:53:...	4184653573199718274	531753237516076726	true
2026-01-07 21:54:...	5125223829492177301	4184653573199718274	true
2026-01-07 21:54:...	6844673237417600305	5125223829492177301	true
2026-01-07 21:54:...	6634828203344518093	6844673237417600305	true
2026-01-07 21:55:...	7637728273407236194	6634828203344518093	true
2026-01-07 21:55:...	3424071684958740192	7637728273407236194	true
2026-01-07 21:55:...	1743746294196424254	3424071684958740192	true

系統會顯示代表不同快照 ID 的資料列，以及這些快照的提交時間。

比較目前與過去的資料列數

建立名為 `timetravel.py` 的檔案：

```
cat <<EOF > timetravel.py
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

query = """
SELECT 'Current State' AS version, COUNT(*) AS count FROM public_data.nyc_taxicab
UNION ALL
SELECT 'Past State' AS version, COUNT(*) AS count FROM public_data.nyc_taxicab VERSION AS OF
4573466015237516024
"""

spark.sql(query).show()
EOF
```

然後提交：

```
gcloud dataproc batches submit pyspark \  
  --project=$PROJECT_ID \  
  --region=$REGION \  
  --version=2.3 \  
  --properties="$METASTORE_PROPERTIES" \  
  --deps-bucket=gs://$DEPS_BUCKET \  
  timetravel.py
```

控制台應會顯示類似下列內容的輸出結果：

```
+-----+-----+  
|      version|      count|  
+-----+-----+  
|Current State|1293069366|  
|  Past State|  72878594|  
+-----+-----+
```

確保您可以稽核一段時間內的資料變更。

步驟 6 · 約 10 分鐘

6. 使用 BigQuery ML 進行結構化 AI

您已探索 Iceberg 資料，現在來使用 BigQuery AI 功能吧！由於公開 Iceberg 目錄為唯讀，我們可以透過 BigQuery 從公開資料表讀取資料，在工作區中訓練模型。

建立本機資料集

首先，請使用 `bq` CLI 在專案中建立資料集，以便儲存 AI 模型：

```
bq mk --location=$REGION --project_id=$PROJECT_ID iceberg_ai
```

訓練線性迴歸模型

現在，您將使用公開的 BigLake Iceberg 資料表訓練線性迴歸模型。

建立查詢檔案，並使用 `bq query` 訓練模型：

```
cat <<'EOF' > train_model.sql
CREATE OR REPLACE MODEL `iceberg_ai.predict_fare`
OPTIONS(model_type='LINEAR_REG', input_label_cols=['fare_amount']) AS
SELECT fare_amount, passenger_count, CAST(trip_distance AS FLOAT64) AS trip_distance
FROM `bigquery-public-data`.`biglake-public-nyc-taxi-iceberg`.public_data.nyc_taxicab
WHERE fare_amount > 0 AND trip_distance > 0 AND RAND() < 0.01; -- Using 1% of data to
downsample
EOF

bq query --location=$REGION --use_legacy_sql=false < train_model.sql
```

`RAND() < 0.01` 是一種縮減技術，可快速訓練模型，同時維持低資源用量！

您可能會收到警告，指出 `passenger_count` 資料欄包含空值。BigQuery ML 會在訓練期間自動處理 NULL 值 (透過插補)，因此您可以放心繼續操作。詳情請參閱「[BigQuery ML 中的插補](#)」。

使用模型預測

模型訓練完成後，您可以使用 `ML.PREDICT` 預測新行程的車資。

建立查詢檔案，並使用 `bq query` 執行預測：

```
cat <<'EOF' > predict_fare.sql
SELECT
  predicted_fare_amount, passenger_count, trip_distance
FROM
  ML.PREDICT(MODEL `iceberg_ai.predict_fare`,
    (
      SELECT 2 AS passenger_count, 5.0 AS trip_distance
    )
  );
EOF

bq query --location=$REGION --use_legacy_sql=false < predict_fare.sql
```

輸出結果應該會類似下列內容：

predicted_fare_amount	passenger_count	trip_distance
14.12252095150709	2	5.0

7. 搭配 BigLake 使用 Unstructured AI

資料不只是資料列和資料欄。統一資料湖倉也能處理非結構化資料 (圖片、PDF)。我們將使用物件資料表和物件參照查詢非結構化資料。

物件資料表是 BigQuery 中的唯讀外部資料表，會列出 Cloud Storage 路徑中的物件。每一列代表一個檔案，其中包含 `uri`、`size` 等中繼資料的資料欄，以及包含 `ObjectRef` 的特殊 `ref` 資料欄。

物件參照 (`ObjectRef`) 會指向單一檔案的實際資料。新版 BigQuery ML 函式 (例如 `AI.GENERATE` 或 `AI.AGG`) 會耗用 `ObjectRef` 讀取檔案內容 (圖片、音訊或文字) 以進行分析，不必將位元組載入標準資料表。

建立非結構化 AI 資料集

首先，在專案中建立第二個資料集，使用 `bq` CLI 在 `us` 多區域中儲存物件資料表：

```
bq mk --location=US --project_id=$PROJECT_ID iceberg_object_ai
```

建立外部連線

如要從 BigQuery 查詢儲存在 Cloud Storage 中的資料 (包括物件資料表和非結構化資料)，您需要建立外部連線。

在 Cloud Shell 中執行下列指令，建立 Cloud 資源連結：

```
bq mk --connection --project_id=$PROJECT_ID --location=US --connection_type=CLOUD_RESOURCE my-conn
```

找出為連線建立的服務帳戶 ID：

```
CONNECTION_SA=$(bq show --format=json --project_id=$PROJECT_ID --connection $PROJECT_ID.us.my-conn | jq -r '.serviceAccountId // .cloudResource.serviceAccountId')
```

將「Vertex AI 使用者」和「Storage 物件檢視者」角色授予服務帳戶，讓服務帳戶可以呼叫 Gemini 模型及讀取 GCS 資料：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$CONNECTION_SA" \
  --role="roles/aiplatform.user"

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$CONNECTION_SA" \
  --role="roles/storage.objectViewer"
```

建立物件資料表

我們將使用上一節建立的外部連線 `my-conn`，存取非結構化資料。建立查詢檔案，並使用 `bq query` 建立物件資料表：

```
cat <<'EOF' > create_object_table.sql
CREATE EXTERNAL TABLE `iceberg_object_ai.sample_images`
WITH CONNECTION `us.my-conn`
OPTIONS (
  object_metadata = 'SIMPLE',
  uris = ['gs://cloud-samples-data/vision/landmark/*']
);
EOF

bq query --use_legacy_sql=false < create_object_table.sql
```

使用 Gemini 處理物件資料

現在，您可以使用 Gemini 執行查詢，評估圖片內容，不必下載圖片！

透過 `bq query` 使用標準 SQL 查詢圖片：

```
cat <<EOF > query_images.sql
SELECT
  uri,
  image_analysis.description
FROM (
  SELECT
    uri,
    AI.GENERATE(
      (
        'Identify what is happening in the image.',
        ref
      ),
      connection_id => 'us.my-conn',
      endpoint => 'gemini-2.5-flash-lite',
      output_schema => 'event STRING, severity STRING, description STRING'
    ) AS image_analysis
  FROM
    iceberg_object_ai.sample_images
);
EOF

bq query --use_legacy_sql=false < query_images.sql
```

輸出內容範例：

uri	description
gs://cloud-samples-data/vision/landmark/eiffel_tower.jpg	The Eiffel Tower stands tall against a cloudy sky, overlooking the Seine River in Paris. Boats are docked along the riverbank, and trees line the opposite shore, with bridges and buildings visible in the distance.
gs://cloud-samples-data/vision/landmark/pofa.jpg	A wide shot shows the Palace of Fine Arts, a monumental structure in San Francisco, California. The building features a large rotunda with a dome, surrounded by colonnades. In front of the rotunda is a lagoon. Several people are walking around the grounds. The sky is blue with a few scattered clouds.
gs://cloud-samples-data/vision/landmark/st_basils.jpeg	A monument stands in front of Saint Basil's Cathedral in Moscow under a bright blue sky with scattered white clouds. The cathedral features distinctive onion domes in various colors and patterns, including red, blue and white stripes, green and beige stripes, and red and blue diamonds. A large green tree partially obscures the left side of the cathedral. People are visible in the foreground near the base of the monument and the cathedral entrance.

直接探索 ObjectRef：情緒分析

物件資料表會自動管理檔案參照，但您可以使用 BigQuery 物件參照直接與這些物件互動，對單一檔案執行即時分析。

舉例來說，您可以使用儲存在自己 GCS bucket 中的小型文字檔 (使用先前建立的 `$DEPS_BUCKET` 變數)，並透過 `OBJ.MAKE REF` 和 `bq query` 分析該檔案。

首先，請建立小型文字檔案並上傳至 bucket：

```
cat <<'EOF' > review.txt
This product is fantastic! It exceeded my expectations. The quality is top-notch. I highly
recommend it to everyone!
EOF

gcloud storage cp review.txt gs://${DEPS_BUCKET}/review.txt
```

現在使用標準 SQL 中的 `OBJ.MAKE_REF` 查詢檔案：

```
cat <<EOF > sentiment_analysis.sql
SELECT
  AI.GENERATE(
    (
      'Analyze the sentiment of this text file. Is it positive, negative, or neutral? Explain
why.',
      OBJ.MAKE_REF('gs://${DEPS_BUCKET}/review.txt', 'us.my-conn')
    ),
    connection_id => 'us.my-conn',
    endpoint => 'gemini-2.5-flash-lite'
  ).result AS ml_generate_text_result;
EOF

bq query --use_legacy_sql=false < sentiment_analysis.sql
```

輸出內容範例：

```
+-----+
+-----+
-+
|
ml_generate_text_result
|
+-----+
+-----+
-+
| This text file has a **strongly positive** sentiment.
|
|
|
| Here's why:
|
|
|
| * **Positive Keywords:** The text is filled with unequivocally positive words and
phrases:
|
| * "fantastic"
|
| * "exceeded my expectations"
|
| * "top-notch"
|
| * "highly recommend"
|
|
|
| * **Enthusiastic Language:** The use of exclamation marks ("!") further amplifies the
positive tone, indicating excitement and strong approval.
|
|
|
| * **Lack of Negative or Neutral Elements:** There are no words, phrases, or implications
that suggest any dissatisfaction, criticism, or even indifference.
|
|
|
| In summary, the author's language is enthusiastic and uses multiple strong positive
```

descriptors, leaving no room for doubt that their opinion of the product is overwhelmingly positive. |

+-----

-+

步驟 8 · 約 5 分鐘

8. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收取費用，請刪除在本程式碼研究室中建立的資源。

刪除資料集和連線

在 Cloud Shell 中執行下列指令，刪除資料集和連線：

```
bq rm -r -f --location=$REGION iceberg_ai
bq rm -r -f --location=US iceberg_object_ai
bq rm --connection $PROJECT_ID.US.my-conn
```

刪除 GCS 值區和本機檔案

清理 GCS bucket 和本機檔案：

```
# Delete GCS buckets
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format="value(projectNumber)")
gcloud storage rm -r gs://dataproc-temp-${REGION}-${PROJECT_NUMBER}-*
gcloud storage rm -r gs://dataproc-staging-${REGION}-${PROJECT_NUMBER}-*
gcloud storage rm -r gs://${DEPS_BUCKET}

# Delete local files
rm -f train_model.sql predict_fare.sql create_object_table.sql query_images.sql
sentiment_analysis.sql test.py query.py history.py timetravel.py review.txt
```

如果您專為本實驗室建立了專案，也可以刪除整個專案。

步驟 9 · 約 0 分鐘

9. 恭喜

恭喜！您已成功使用 Apache Iceberg、BigLake 和 BigQuery AI 建構整合式資料湖倉！

目前所學內容

- 如何連線及查詢公開 **Apache Iceberg** REST 目錄。
- 在 Iceberg 中使用「時間旅行」稽核資料集版本。
- 使用結構化資料訓練 **BigQuery ML** 模型。
- 使用物件資料表和 ObjectRef 連結非結構化資料 (圖片)。
- 直接在 **BigQuery SQL** 中使用 **Gemini** 分析圖片。

參考文件

- [BigLake 總覽](#)
 - [BigQuery ML 生成式 AI](#)
 - [Apache Iceberg 網站](#)
-